

Lab 23: Distance-Weighted KNN

Aim

To implement **Distance-Weighted K-Nearest Neighbors (KNN)** for classification, where closer neighbors have a greater influence on the prediction.

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Create a KNN classifier with distance-based weights.
5. Train the model using the training data.
6. Predict the classes for the test data.
7. Calculate and display the accuracy.

Program

```
# Lab 23: Distance-Weighted KNN
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
# Load the Iris dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Create Distance-Weighted KNN classifier
model = KNeighborsClassifier(
    n_neighbors=3,
    weights="distance"
)
```

```
# Train the model
model.fit(X_train, y_train)
```

```
# Make predictions
y_pred = model.predict(X_test)
```

```
print("Predictions:", y_pred)
print("Actual Values:", y_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Sample Output

```
Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 1 0 2 2 2 2 2 0 0]
```

```
Actual Values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 1 0 2 2 2 2 2 0 0]
```

```
Accuracy: 1.0
```

Result

Thus, the **Distance-Weighted KNN algorithm was successfully implemented** using the Iris dataset, where nearer data points were given greater importance during classification.